

**CENTRO ESTADUAL DE EDUCAÇÃO TECNOLÓGICA PAULA SOUZA FACULDADE
DE TECNOLOGIA DE SOROCABA JOSÉ CRESPO GONZALES**

Análise e Desenvolvimento de sistemas

Programação Web

THAÍS DUARTE DE SOUZA

RA: 0030482511015

APIs E INTEGRAÇÕES

SOROCABA – SP

2026

SUMÁRIO

1. INTRODUÇÃO.....	3
2. APIS E INTEGRAÇÃO DE SISTEMAS.....	4
2.1. Conceito de API.....	4
2.2. Comunicação entre sistemas.....	5
2.3. Importância das APIs.....	6
3. TIPOS DE APIS.....	7
3.1. REST.....	7
3.2. SOAP.....	8
3.3. GraphQL.....	8
3.4. Comparação entre tipos.....	8
4. INTEGRAÇÃO DE SISTEMAS POR APIS.....	9
5. FERRAMENTAS E TECNOLOGIAS.....	10
5.1. Desenvolvimento.....	10
5.2. Testes.....	10
5.3. Documentação.....	10
5.4. Gerenciamento e DevOps.....	11
6. EXEMPLOS DE UTILIZAÇÃO.....	11
7. VANTAGENS, LIMITAÇÕES E DESAFIOS.....	13
7.1. Vantagens.....	13
7.2. Limitações.....	13
7.3. Segurança.....	14
7.4. Manutenção e versionamento.....	15
8. BOAS PRÁTICAS.....	16
9. CONCLUSÃO.....	17
REFERÊNCIAS.....	17

1. INTRODUÇÃO

No cenário contemporâneo da engenharia de *software*, a descentralização de funcionalidades e o crescimento das interdependências tecnológicas tornaram a integração de sistemas um aspecto vital para a conectividade operacional das empresas. Historicamente, o desenvolvimento de sistemas era restrito a aplicações monolíticas cujas funções de apresentação, regras de negócios e repositórios de dados coexistiam em uma única camada física. A alta complexidade de manutenção e a rigidez dessas estruturas impulsionaram a transição para aplicações multicamadas (cliente-servidor, arquiteturas de três ou N camadas), nas quais a separação lógica de responsabilidades e a introdução de *Web Services* permitiram a cooperação entre máquinas geograficamente dispersas e baseadas em diferentes sistemas operacionais e linguagens.

A redundância na lógica de negócios e o acoplamento rígido eram comuns nessas abordagens legadas (Fenerich, 2023). A Arquitetura Orientada a Serviços (SOA) surgiu como uma resposta tática a esse cenário de fragmentação, propondo o reuso de serviços distribuídos na rede para reduzir a duplicação lógica (Fenerich, 2023). Nesse processo evolutivo, as Interfaces de Programação de Aplicativos (APIs) consolidaram-se como o principal mecanismo para viabilizar a troca de dados e a orquestração de serviços entre plataformas heterogêneas (Novais; Stekel, 2025).

Empresas líderes de tecnologia e órgãos públicos passaram a expor seus ativos e dados valiosos por meio de APIs, permitindo que parceiros e desenvolvedores externos criem aplicativos inovadores de forma ágil, transformando processos industriais, logísticos e de governança eletrônica (Jacobson; Brail, 2011; Suzuki; Romualdo-Suzuki, 2020). Dessa forma, as APIs atuam como verdadeiros ativos contratuais e sócio-técnicos estruturantes (De Souza; Redmiles, 2009).

O objetivo deste artigo é analisar conceitualmente as APIs e seus padrões de integração, descrevendo os estilos arquiteturais prevalentes, as tecnologias e ferramentas de suporte ao desenvolvimento, testes e documentação, bem como os

desafios sócio-técnicos de segurança, manutenção e versionamento no ciclo de vida do software. A organização do trabalho segue uma estrutura sistemática: a Seção 2 aborda os conceitos e a importância das APIs; a Seção 3 detalha e compara os principais tipos de APIs baseando-se em métricas de uso mercadológico; a Seção 4 discute as arquiteturas e os modelos de integração; a Seção 5 expõe as ferramentas de desenvolvimento, teste e documentação; a Seção 6 traz exemplos e estudos de caso; a Seção 7 examina vantagens, limitações, segurança e versionamento; a Seção 8 sumariza as boas práticas extraídas da literatura; e a Seção 9 apresenta as considerações finais do estudo.

2. APIS E INTEGRAÇÃO DE SISTEMAS

2.1. Conceito de API

Uma Interface de Programação de Aplicativos (API) refere-se a um conjunto de especificações de interfaces, classes, métodos e protocolos definidos que permitem que uma aplicação de *software* acesse programaticamente serviços ou dados fornecidos por outro componente de *software*, sem a necessidade de compreender a complexidade interna de sua implementação de código. Em termos práticos, as APIs atuam como um contrato de serviço firmado entre duas plataformas de *software*: o aplicativo que emite uma solicitação é denominado 'cliente', enquanto a infraestrutura que fornece a resposta e processa os dados é conhecida como 'servidor' (Alves, 2021).

Este contrato estabelece um comportamento previsível e consistente para as interações. Sob a ótica sócio-técnica da engenharia de *software*, as APIs instalam diretamente o princípio do ocultamento de informações (*information hiding*) formulado originalmente por Parnas em 1972 (De Souza; Redmiles, 2009). Sob essa diretriz, os módulos expõem exclusivamente uma interface estável e mantêm ocultos os detalhes de baixo nível e as estruturas de bancos de dados que estão propensos a sofrer modificações frequentes (De Souza; Redmiles, 2009).

2.2. Comunicação entre sistemas

A comunicação entre sistemas por meio de APIs baseia-se no tráfego de mensagens estruturadas através de redes físicas ou da Internet, utilizando formatos comuns que ambas as partes compreendam (Alves, 2021). No início do desenvolvimento da *Web*, as trocas dependiam de textos informais e, posteriormente, de formatos baseados em *XML* (como no padrão *SOAP*) (Neumann; Laranjeiro; Bernardino, 2018). Atualmente, o formato *JSON* (*JavaScript Object Notation*) é o padrão dominante de fato por ser um formato em texto puro facilmente legível por humanos e processado de forma eficiente por máquinas em redes heterogêneas (Neumann; Laranjeiro; Bernardino, 2018).

Um aspecto crítico da comunicação entre sistemas locais legados e APIs externas é a necessidade de compatibilização e mapeamento de campos de dados (Novais; Stekel, 2025). Sistemas locais operam sob configurações internas específicas, cujos dados precisam ser convertidos antes da transmissão para o formato esperado pela API consumida. Por exemplo, campos numéricos locais (como 'cliente_id' e 'status') precisam ser mapeados e traduzidos para cadeias de texto ou formatos padronizados internacionalmente, como datas em conformidade com a norma ISO 8601, de modo a garantir a integridade e evitar falhas de comunicação (Novais; Stekel, 2025).

Tabela 1. Mapeamento de correspondência de dados locais para API externa.

Campos Locais do Sistema Interno	Formato Padrão Local	Campo da API Externa	Formato Esperado pela API
cliente_id	Inteiro (integer)	customerID	Texto (string)

data_pedido	Data e hora (datetime)	orderDate	Formato ISO 8601 (yyyy-MM-ddTHH: mm:ssZ)
status	Inteiro (0 = pendente; 1 = fechado)	orderStatus	Texto (220061 = pendente; 220062 = fechado)

Fonte: Adaptado de Novais e Stekel (2025).

2.3. Importância das APIs

As APIs tornaram-se elementos estratégicos para as organizações ao permitirem que diferentes sistemas colaborem em tempo real, impulsionando a eficiência operacional e reduzindo drasticamente custos logísticos, operacionais e de impressão (Suzuki; Romualdo-Suzuki, 2020). No contexto do desenvolvimento de *software* distribuído, as *APIs* exercem três papéis coordenativos fundamentais, representados por metáforas teóricas na literatura de engenharia de *software* (De Souza; Redmiles, 2009):

- **1. APIs como Contratos:** Elas estabelecem uma especificação estável que permite o avanço independente do design interno de cada componente, promovendo o desenvolvimento em paralelo entre a equipe que consome a API (como programadores frontend) e a equipe que a implementa no servidor (desenvolvedores backend).
- **2. APIs como Fronteiras (Boundaries):** Delimitam de forma clara o escopo de atuação técnica e organizacional de cada equipe, reduzindo a sobrecarga cognitiva e permitindo que os times foquem exclusivamente na resolução de problemas de seus domínios específicos.
- **3. APIs como Mecanismos de Comunicação:** Simplificam as discussões técnicas diárias entre programadores ao estabelecerem um 'terreno comum'

formalizado composto por classes, assinaturas e estruturas bem definidas, facilitando o diagnóstico de falhas.

Esses mecanismos coordenativos reduzem a necessidade de comunicação presencial constante, o que permite escalar as equipes organizacionais de forma eficiente, mitigando atrasos em projetos complexos (De Souza; Redmiles, 2009).

3. TIPOS DE APIS

3.1. REST

O estilo arquitetural de Transferência de Estado Representacional (*REST*) foi formulado por Roy Thomas Fielding em sua tese de doutorado no ano de 2000 (Fielding, 2000). O *REST* não se constitui como um padrão de especificação rígido ou protocolo, mas sim como um conjunto de restrições de *design* voltadas para otimizar o desempenho, a escalabilidade, a simplicidade e a confiabilidade de sistemas de hipermídia distribuídos na *World Wide Web* (Fielding, 2000; Neumann; Laranjeiro; Bernardino, 2018). O estilo baseia-se em restrições fundamentais (Fielding, 2000):

- **Separação Cliente-Servidor:** Desacopla a interface de usuário do armazenamento de dados, permitindo a evolução independente das plataformas.
- **Ausência de Estado (*Statelessness*):** Cada requisição do cliente para o servidor deve conter todas as informações necessárias para sua compreensão e processamento isolado, sem depender de sessões armazenadas no *backend*.
- **Cacheabilidade (*Cache*):** As respostas do servidor devem se autodeclarar explicitamente como cacheáveis ou não, otimizando o tráfego de rede.
- **Interface Uniforme:** Estabelece um contrato padronizado que simplifica a arquitetura global, baseado na identificação única de recursos por *URIs*, manipulação por representações, mensagens autodescritivas e o uso de *HATEOAS* (*Hypermedia As The Engine Of Application State*).

- **Sistema em Camadas:** Permite o uso de intermediários físicos na rede, como gateways de balanceamento de carga, caches e proxies, que interceptam as requisições de forma transparente ao cliente.

3.2. SOAP

Diferente do *REST*, o *SOAP* (*Simple Object Access Protocol*) é um protocolo estruturado e formalmente padronizado pelo *W3C*, que utiliza mensagens estritamente formatadas em *XML* (Neumann; Laranjeiro; Bernardino, 2018). O *SOAP* depende da especificação de contratos semânticos rígidos por meio de arquivos *WSDL* (*Web Services Description Language*), que determinam exatamente as operações e tipos de dados aceitos (Neumann; Laranjeiro; Bernardino, 2018). Devido ao peso computacional do processamento *XML* e ao alto acoplamento induzido por seus contratos rígidos, o *SOAP* apresenta menor flexibilidade que o estilo *REST*, sendo relegado a nichos como transações financeiras e integração de sistemas legados que exigem conformidade com o padrão *WS-Security* (Jacobson; Brail, 2011).

3.3. GraphQL

O *GraphQL* surgiu no cenário tecnológico recente para preencher lacunas de desempenho associadas a APIs *RESTful* em ecossistemas de dados altamente conectados (Singh; Yugandhar; Chawla, 2024). Enquanto o estilo *REST* pode incorrer no problema de sobrefaturamento de dados (*over-fetching*), em que uma única requisição a um *endpoint* genérico retorna mais propriedades do que as necessárias para a visualização no cliente, o *GraphQL* introduz uma linguagem de consulta flexível (Singh; Yugandhar; Chawla, 2024). Por meio dela, o aplicativo cliente define exatamente quais campos e entidades aninhadas deseja obter, e o servidor responde em uma única requisição, otimizando o uso de banda e reduzindo a latência (Singh; Yugandhar; Chawla, 2024).

3.4. Comparação entre tipos

A análise empírica de repositórios de APIs públicas revela uma preferência esmagadora da indústria pelo padrão *REST*, em virtude de sua menor curva de

aprendizado e suporte nativo ao protocolo *HTTP* (Neumann; Laranjeiro; Bernardino, 2018). Pesquisas indicam que mais de 70% das APIs expostas publicamente adotam os princípios de *design* do *REST* (Neumann; Laranjeiro; Bernardino, 2018). O *SOAP* mantém presença estável em cerca de 10% a 26% das integrações corporativas legadas devido à sua robustez transacional, enquanto o *GraphQL* apresenta crescimento contínuo em aplicações móveis e ricas em dados (Neumann; Laranjeiro; Bernardino, 2018; WJAETS, 2024).

4. INTEGRAÇÃO DE SISTEMAS POR APIS

A necessidade de conectar múltiplos sistemas corporativos deu origem a diferentes abordagens arquiteturais ao longo das décadas. A Arquitetura Orientada a Serviços (SOA) consolidou o conceito de modularização de processos de negócios por meio da exposição de serviços compartilhados na rede, visando mitigar redundâncias de implementação e reduzir a complexidade de manutenção (Fenerich, 2023). Tradicionalmente, a orquestração desses serviços em ambientes corporativos de grande porte baseava-se em Barramentos de Serviços Corporativos (*ESB*), intermediários centralizados que realizavam a transformação de mensagens e protocolos entre sistemas heterogêneos (Silva; Justino; Adachi, 2022).

Com o advento das práticas *DevOps* e a demanda por escalabilidade massiva, as organizações passaram a evoluir os conceitos da SOA em direção à arquitetura de microsserviços (Silva; Justino; Adachi, 2022). Sob esse modelo, as aplicações monolíticas são decompostas em serviços independentes e altamente especializados, que governam seus próprios repositórios de dados e se comunicam por meio de APIs *RESTful* leves ou mensageria assíncrona (Silva; Justino; Adachi, 2022). Na estruturação interna desses sistemas, aplicam-se padrões de projeto como *CQRS* (*Command Query Responsibility Segregation*) para segregar fluxos de escrita e leitura, Camadas *Anti-Corrupção* (*ACL*) para proteger os modelos internos e o padrão *Sidecar* para abstrair responsabilidades transversais de infraestrutura (Silva; Justino; Adachi, 2022).

5. FERRAMENTAS E TECNOLOGIAS

5.1. Desenvolvimento

A literatura científica documenta o uso de uma ampla variedade de ferramentas e plataformas para a construção e execução de APIs. Em ambientes corporativos integrados, destaca-se o uso de soluções baseadas em tecnologia C# (.NET Framework e ASP.NET Web API) gerenciadas pelo *Microsoft Visual Studio*, operando conexões com repositórios *Oracle 10g* via transações *PL/SQL* (Suzuki; Romualdo-Suzuki, 2020). Em paralelo, no domínio das APIs semânticas e sistemas de conhecimento de bioinformática, as especificações *Java JAX-RS (Java API for RESTful Web Services)* ditam a criação de classes anotadas (Guardia et al., 2012). No ecossistema de microsserviços modernos, sobressaem também *frameworks* ágeis como *Spring Boot*, *Node.js* e *Express* (Silva; Justino; Adachi, 2022).

5.2. Testes

Os testes de APIs constituem uma etapa altamente complexa, uma vez que as regras de negócios do *backend* estão ocultas para o consumidor da interface (Ehsan et al., 2022). Estreantes em engenharia de *software* estimam que mais de 35% das vulnerabilidades operacionais decorrem de validações insuficientes nas chamadas de dados (Ehsan et al., 2022). Para lidar com esses desafios, ferramentas modernas executam testes de caixa-preta automatizados guiados por restrições de formato (como no utilitário *RESTest*) (Ehsan et al., 2022). Outra linha de pesquisa busca derivar suítes de teste de unidade diretamente da análise estática das especificações formais de interface, garantindo ampla cobertura do código de produção (Ehsan et al., 2022).

5.3. Documentação

A documentação é um pilar crítico para a adoção sustentável de APIs, permitindo que programadores externos compreendam as capacidades expostas sem necessidade de acessar o código-fonte (Casas et al., 2021). A especificação *OpenAPI* (antigo *Swagger UI*) consolidou-se como o padrão de fato da indústria, sendo empregada para a geração automática de painéis interativos em cerca de 45% das APIs e influenciando

diretamente 43% dos processos de melhoria de qualidade de documentação pesquisados (Casas et al., 2021). Outras ferramentas comuns de visualização incluem *Apiary*, *Apigee SmartDocs*, *Postman* e *RAMLtoHTML* (Casas et al., 2021). No campo dos serviços semânticos, utilizam-se anotações *SAWSDL* estruturadas no ambiente *Eclipse* via *Radiant* (Guardia et al., 2012).

5.4. Gerenciamento e DevOps

Na arquitetura de microsserviços distribuídos, o gerenciamento unificado do tráfego e as políticas de segurança são centralizados em *API Gateways* (como o *Netflix Zuul*) que atuam como intermediários físicos de borda (Silva; Justino; Adachi, 2022). Os *gateways* tratam requisições transversais como limitação de taxa (*rate limiting*), conversão de cabeçalhos e controle de acessos (Silva; Justino; Adachi, 2022). Sob a perspectiva de *DevOps*, o monitoramento contínuo da integridade das chamadas de rede combina o uso de barramentos de mensagens (como o *RabbitMQ*) com a pilha *ELK* (*Elasticsearch*, *Logstash* e *Kibana*) para centralizar *logs* e gerar painéis analíticos de telemetria em tempo real (Silva; Justino; Adachi, 2022).

6. EXEMPLOS DE UTILIZAÇÃO

As fontes documentam diversos casos práticos que ilustram o impacto operacional das integrações de sistemas por meio de APIs:

Caso 1: CT-e e Sistemas de Gestão de Transportes (SEFAZ). Antes da implantação do projeto Conhecimento de Transporte Eletrônico (CT-e) pelo governo federal brasileiro, as transportadoras realizavam a validação física de guias em papel, o que gerava severos atrasos logísticos nas fronteiras estaduais (Suzuki; Romualdo-Suzuki, 2020). Para resolver a disparidade técnica entre o sistema corporativo interno legado da transportadora (desenvolvido em *COBOL*) e a infraestrutura dos *Web Services* governamentais em *XML* (SEFAZ), a transportadora desenvolveu uma arquitetura de integração em três camadas com tecnologias *C# .NET* e banco *Oracle 10g* (Suzuki; Romualdo-Suzuki, 2020). Uma aplicação do tipo *Windows Service* faz a leitura de arquivos texto (*.TXT*) locais gerados pelo legado, agrupa de forma síncrona ou assíncrona lotes de até 50 conhecimentos de

transporte e os converte para o formato *XML* esperado pela SEFAZ. O processo eliminou custos de impressão e reduziu drasticamente o tempo de parada dos caminhões em postos fiscais (Suzuki; Romualdo-Suzuki, 2020).

Caso 2: Reengenharia de Sistemas Fiscais (SET-RN). A Secretaria de Tributação do Estado do Rio Grande do Norte (SET-RN) possuía um sistema central de tributação crítico (*UVT*) que acumulava dívida técnica severa e altos custos de manutenção (Silva; Justino; Adachi, 2022). Para reestruturá-lo, a instituição aplicou o processo *SPReaD* (método orientado a serviços integrado a *DevOps*), decompondo o monólito legado em diversos contextos de microsserviços independentes estruturados em *C#/NET* e encapsulados em *containers Docker*, gerenciados por *API Gateway* nas bordas e monitorados pela pilha *ELK*. A nova arquitetura aumentou substancialmente os níveis de desempenho e escalabilidade da infraestrutura fiscal do estado, além de padronizar as metodologias de *release* de código da equipe técnica (Silva; Justino; Adachi, 2022).

Caso 3: Repositório Semântico GEAS para Genômica. Pesquisadores na área de bioinformática enfrentavam limitações em virtude de plataformas de análise genômica rígidas e difíceis de estender (Guardia et al., 2012). Para solucionar essa barreira, foi estruturado o repositório *GEAS* (*Gene Expression Analysis Services*), construindo *Web Services* semânticos *RESTful* encapsulados por metodologias *Java JAX-RS* e descritos semanticamente por ontologias biológicas ricas desenvolvidas em *OWL* (*GEXPO* e *GEXPASO*) (Guardia et al., 2012). As anotações do *WSDL 2.0* por meio de *Radiant* no *Eclipse* permitiram o mapeamento formal de operações complexas, facilitando a automação de consultas por bioinformatas (Guardia et al., 2012).

Caso 4: Implementações em Escala Global. Empresas globais utilizam APIs para sustentar modelos de negócios de alta escalabilidade (Jacobson; Brail, 2011). A Netflix migrou seu monólito para centenas de microsserviços na nuvem AWS, apoiando-se no *Zuul* como *API Gateway*, *Eureka* para descoberta de serviços, *Hystrix* para tolerância a falhas e *EVCache* para *cache* em memória, obtendo resiliência diante de milhões de acessos concorrentes (Jacobson; Brail, 2011). A

NPR desenvolveu uma API para desacoplar seu site de um ponto único de falha em banco de dados *Oracle*, migrando para *MySQL* de forma transparente e permitindo que afiliadas fizessem *mashups* dinâmicos de conteúdo (Jacobson; Brail, 2011). No *eBay* e na *Salesforce*, as APIs superaram os acessos diretos de seus websites principais, sendo responsáveis por mais de 60% de todas as transações globais da *Salesforce* (Jacobson; Brail, 2011). Empresas como Innotas, Sears e Twitter também abriram seus catálogos e serviços via APIs, estendendo suas cadeias de distribuição *B2B* (Jacobson; Brail, 2011).

7. VANTAGENS, LIMITAÇÕES E DESAFIOS

7.1. Vantagens

As integrações baseadas em APIs proporcionam expressivo valor tecnológico e estratégico para as organizações. Entre as vantagens mais documentadas destacam-se a interoperabilidade e o reuso de código, permitindo a cooperação entre plataformas estruturadas em diferentes sistemas de *hardware* e linguagens de programação geograficamente dispersas (Alves, 2021; Novais; Stekel, 2025). Além disso, as APIs reduzem custos operacionais e administrativos com a eliminação do trâmite manual de guias e do armazenamento físico de papéis (Suzuki; Romualdo-Suzuki, 2020). Sob o aspecto do gerenciamento de projetos, o estabelecimento de um contrato de interface claro (API) promove o trabalho em paralelo das equipes, desacoplando o desenvolvimento de *frontend* e *backend*. Por fim, elas viabilizam a automação de funções cotidianas de sistema, como a sincronização automática de data e hora de computadores portáteis ao se deslocarem entre fusos horários (De Souza; Redmiles, 2009).

7.2. Limitações

Apesar de seus indiscutíveis benefícios, a dependência severa de APIs introduz desafios socio-técnicos complexos (De Souza; Redmiles, 2009). Ao estabelecer limites contratuais formais e muito rígidos entre os módulos, cria-se nas equipes a 'ilusão de independência completa' (De Souza; Redmiles, 2009). À medida que os contatos diretos entre os engenheiros diminuem e a comunicação passa a depender

exclusivamente da especificação da API, reduzem-se as interações informais que previnem desalinhamentos operacionais. Isso pode criar lacunas graves de comunicação, onde um time altera um aspecto periférico da API que acarreta falhas críticas imprevistas no outro módulo (De Souza; Redmiles, 2009).

As APIs sofrem com a limitação teórica da 'incompletude inerente' (De Souza; Redmiles, 2009). Embora uma especificação (como *OpenAPI*) consiga detalhar com precisão os aspectos sintáticos da interface (parâmetros, tipos de dados e rotas), ela é incapaz de explicitar por completo a semântica interna do *backend*, o fluxo dinâmico dos estados do sistema e o comportamento da rede. Essa opacidade força os desenvolvedores clientes a realizarem suposições, o que frequentemente resulta em *bugs* difíceis de diagnosticar em ambientes de produção (De Souza; Redmiles, 2009).

7.3. Segurança

A exposição de *endpoints* na *Web* amplia significativamente a superfície de ataque das corporações, tornando a segurança um dos desafios mais graves citados por mais de 35% de profissionais de *software* (Ehsan et al., 2022). Vulnerabilidades críticas incluem injeção de código (*SQL Injection*), quebras de autenticação e ataques de negação de serviço (*DoS*) por meio do envio de *payloads* massivos ou malformados (Ehsan et al., 2022). Para mitigar essas ameaças, as arquiteturas modernas apoiam-se em estratégias de proteção integradas:

- **Criptografia Assimétrica RSA de Alto Nível:** Cifra os *payloads* de dados críticos (como *JSONs* locais de ordens de serviço) antes de trafegarem em rede pública. Utiliza chaves públicas para criptografar no cliente e chaves privadas correspondentes para decifrar no servidor. Essa técnica, fundamentada em conceitos clássicos da teoria dos números por matemáticos como Fermat, Euler e Gauss, garante a confidencialidade e a integridade da informação em trânsito (Castro, 2014; Novais; Stekel, 2025).
- **Padrão de Autorização OAuth:** Estrutura aberta que permite a emissão de *tokens* dinâmicos de acesso temporário, evitando que as credenciais e

senhas dos usuários finais trafeguem em formato legível ou sejam armazenadas por aplicativos clientes de terceiros (Silva; Justino; Adachi, 2022).

- **Inspeção Ativa de Tráfego no API Gateway:** Os *gateways* filtram requisições malformadas e aplicam controle de taxas (*rate limiting*) de chamadas de rede nas bordas da infraestrutura, bloqueando comportamentos anômalos antes que cheguem aos servidores internos (Silva; Justino; Adachi, 2022).

7.4. Manutenção e versionamento

Como as APIs servem de base para o acoplamento de sistemas desenvolvidos por equipes independentes ou parceiros, qualquer alteração imprevista na assinatura de métodos, tipos de dados ou *endpoints* pode causar falhas críticas e imediatas ('quebra') nos clientes que consomem o serviço (Jacobson; Brail, 2011). No entanto, o dinamismo do *software* exige constante evolução. Para coordenar esse ciclo de vida de forma segura, os arquitetos de *software* adotam estratégias estruturadas de versionamento (Jacobson; Brail, 2011):

- **Versionamento na Rota da URI:** Consiste na inclusão explícita da versão do contrato diretamente no caminho de acesso (por exemplo, '/v1/recursos/'). Essa estratégia fornece uma interface previsível, permitindo que novas versões rodem em paralelo enquanto os clientes realizam a migração gradual de seus sistemas (Jacobson; Brail, 2011).
- **Versionamento por Cabeçalho HTTP:** Utiliza os cabeçalhos padrão do protocolo *HTTP* (como 'Accept' ou cabeçalhos proprietários) para negociar dinamicamente a versão específica do recurso desejada pelo cliente (Jacobson; Brail, 2011).
- **Estratégia Versionless (Sem Versões):** Baseia-se em manter compatibilidade total para trás, adicionando apenas campos opcionais ou novos endpoints não mandatórios, o que exige extrema disciplina técnica das equipes (Jacobson; Brail, 2011).

8. BOAS PRÁTICAS

Apoiando-se no corpo de evidências empíricas e estudos de caso que compõem o corpus literário deste trabalho, o desenvolvimento de ecossistemas saudáveis e sustentáveis de APIs baseia-se em cinco boas práticas fundamentais:

- **Design Baseado em Recursos (*Pragmatic REST*):** De acordo com investigações analíticas, mais de 87,8% das *APIs Web* de sucesso estruturam suas rotas em torno de substantivos identificadores de recursos (como '/usuarios/joao') em detrimento do uso de verbos operacionais (como '/usuarios/obterNome'), respeitando a semântica natural do estilo *REST* (Neumann; Laranjeiro; Bernardino, 2018).
- **Adoção Padrão do Formato *JSON*:** Exceto em integrações corporativas que requeiram estrita conformidade transacional sob *SOAP*, deve-se adotar o formato *JSON* como padrão para tráfego de requisições e respostas, otimizando o consumo de banda de rede e facilitando o *parsing* em plataformas móveis (Neumann; Laranjeiro; Bernardino, 2018).
- **Implementação de *Rate Limiting*:** Estabelecer limites formais para o número de chamadas permitidas por usuário ou aplicativo parceiro dentro de uma janela temporal, garantindo a estabilidade e a disponibilidade dos servidores internos contra picos inesperados de tráfego ou ataques *DoS* (Silva; Justino; Adachi, 2022).
- **Disponibilização Precoce de *Mocks* e *Local APIs*:** Fornecer implementações locais fictícias ou *stubs* de forma ágil no início do ciclo de vida do projeto. Essa prática permite que os desenvolvedores *frontend* iniciem a programação das regras visuais e interfaces de usuário em paralelo, sem bloqueios de cronograma decorrentes da codificação do *backend* (De Souza; Redmiles, 2009).
- **Documentação Automatizada e Interativa:** Integrar a geração de documentação ao *pipeline* de desenvolvimento (*DevOps*) por meio de

ferramentas como *OpenAPI Specification/Swagger UI*, disponibilizando consoles interativos (*sandboxes*) para testes rápidos de requisições diretamente na interface de documentação (Casas et al., 2021).

9. CONCLUSÃO

A análise sistemática e integrada das fontes deste *Notebook* científico evidencia que as Interfaces de Programação de Aplicativos (APIs) ultrapassaram o papel de simples conectores técnicos de dados para se posicionarem como componentes contratuais, organizacionais e sócio-técnicos de valor vital para a engenharia de *software* contemporânea. A evolução histórica de *middleware* monolíticos e sistemas baseados em barramentos complexos para microsserviços autônomos e integrados reflete a demanda por escalabilidade, manutenibilidade e reuso ágil em ambientes corporativos distribuídos (Fenerich, 2023; Silva; Justino; Adachi, 2022).

Contudo, a pesquisa demonstra de forma contundente que o sucesso das integrações de sistemas por APIs não reside unicamente na escolha das linguagens ou padrões arquiteturais adotados, mas na implementação disciplinada de processos de governança técnica e humana. É imperativo balancear a agilidade com medidas rigorosas de documentação sistemática via *OpenAPI/Swagger UI*, validação exaustiva por meio de testes automatizados e segurança unificada apoiada em criptografia *RSA* de alto nível e autorização *OAuth* (Casas et al., 2021; Ehsan et al., 2022; Novais; Stekel, 2025).

Sob a ótica social, deve-se reconhecer as limitações de incompletude inerente e as barreiras que as fronteiras rígidas de APIs impõem sobre a cooperação entre equipes (De Souza; Redmiles, 2009). Conclui-se que as APIs devem ser concebidas sob uma abordagem holística e integrada, em que as frentes técnicas e organizacionais estejam em constante sintonia e diálogo, alinhando a arquitetura estável do código à comunicação fluida entre as equipes, viabilizando o sucesso da transformação digital.

REFERÊNCIAS

ALVES, William P. **Banco de Dados: teoria e desenvolvimento**. Rio de Janeiro: Érica, 2021. E-book. ISBN 9788536533759. Acesso em: 16 ago. 2026.

CASAS, Sandra; CRUZ, Diana; VIDAL, Graciela; CONSTANZO, Marcela. **Uses and applications of the OpenAPI/Swagger specification**: a systematic mapping of the literature. Universidad de la Patagonia Austral, Rio Gallegos, Argentina, 2021. Acesso em: 12 ago. 2026.

CASTRO, Felipe Lopes. **Criptografia RSA**: uma abordagem para professores do ensino básico. 2014. Monografia (Licenciatura em Matemática) – Universidade Federal do Rio Grande do Sul, Porto Alegre, 2014. Acesso em: 15 ago. 2026.

DA SILVA, Carlos Eduardo; JUSTINO, Yan de Lima; ADACHI, Eiji. **SPReaD: service-oriented process for reengineering and DevOps**: Developing microservices for a Brazilian state department of taxation. Service Oriented Computing and Applications, London, v. 16, p. 1-16, 2022. Acesso em: 14 ago. 2026.

DE SOUZA, Cleidson R. B.; REDMILES, David F. **On The Roles of APIs in the Coordination of Collaborative Software Development**. Computer Supported Cooperative Work, New York, v. 18, p. 445–475, 2009. Acesso em: 11 ago. 2026.

EHSAN, Adeel; ABUHALIQA, Mohammed Ahmad M. E.; CATAL, Cagatay; MISHRA, Deepti. **RESTful API Testing Methodologies**: Rationale, Challenges, and Solution Directions. Applied Sciences, Basel, v. 12, n. 9, p. 4369, 2022. Acesso em: 16 ago. 2026.

FENERICH, Bruno Munaretti. **Arquitetura Orientada a Serviços e sua integração com Software as a Service**. 2023. Monografia (Tecnologia em Análise e Desenvolvimento de Sistemas) – Faculdade de Tecnologia de São Paulo, Centro Estadual de Educação Tecnológica Paula Souza, São Paulo, 2023. Acesso em: 15 ago. 2026.

FIELDING, Roy Thomas. **Architectural Styles and the Design of Network-based Software Architectures**. 2000. 161 f. Tese (Doutorado em Computação e Ciência da Informação) - University of California, Irvine, 2000. Acesso em: 13 ago. 2026.

GUARDIA, Gabriela D. A.; FERREIRA PIRES, Luís; VÊNCIO, Ricardo Z. N.; MALMEGRIM, Kelen C. R.; DE FARIAS, Cléver R. G. **A Methodology for the Development of RESTful Semantic Web Services for Gene Expression Analysis**. PLOS ONE, San Francisco, v. 7, 2012. Acesso em: 15 ago. 2026.

HEUSER, Carlos A. **Projeto de banco de dados - UFRGS**. V. 4. Porto Alegre: Bookman, 2011. E-book. ISBN 9788577804528. Acesso em: 16 ago. 2026.

JACOBSON, Daniel; BRAIL, Greg. **APIs: A Strategy Guide**. Sebastopol: O'Reilly Media, 2011. Acesso em: 11 ago. 2026.

NEUMANN, Andy; LARANJEIRO, Nuno; BERNARDINO, Jorge. **An Analysis of Public REST Web Service APIs**. IEEE Transactions on Services Computing, New York, v. 14, 2018. Acesso em: 14 ago. 2026.

NOVAIS, Ruth Queiroz da Costa; STEKEL, Tardelli Ronan Coelho. **Integração Escalável e Segura de Sistemas de Gestão com APIs: Uma Abordagem Prática.** Revista Foco, Jacareí, v. 18, n. 2, p. 1-10, 2025. Acesso em: 15 ago. 2026.

SINGH, Deepak; YUGANDHAR, Manoj Babu Devapathni; CHAWLA, Nikita. **Design and Implementation Strategies for Scalable RESTful APIs in Enterprise Systems.** World Journal of Advanced Engineering Technology and Sciences, v. 13, n. 1, p. 1045–1047, 2024. Acesso em: 12 ago. 2026.

SUZUKI, Thiago; ROMUALDO-SUZUKI, Larissa. **On the benefit of 3-tier SOA architecture promoting information sharing among TMS systems and Brazilian e-Government Web Services: A CT-e case study.** Proc. 1st Conf. on Computational Interdisciplinary, São Carlos, Brazil, 2020. Acesso em: 13 ago. 2026.

TAQUIWI, Gustavo Ariosi Silva; DIAS, Lucas Silva; BARBOSA, Giulio Venicius Batista. **Integração de API com Banco de Dados e seus Desafios.** Caderno IC 2024, Univale, p. 80-84, 2024. Acesso em: 16 ago. 2026.

WORLD JOURNAL OF ADVANCED ENGINEERING TECHNOLOGY AND SCIENCES. **Application Programming Interfaces (APIs).** WJAETS, v. 13, n. 1, p. 1045–1047, 2024. Acesso em: 14 ago. 2026.